Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Tarih: 15 Mart 2023 Çarşamba. Online ders (Zoom). Konu: MapReduce tekrarı ve iyileştirmeleri (MMDS 2. bölüm), ardından 3. bölüm "Benzer Öğeleri Bulma" (Finding Similar Items): shingling ve MinHashing. Anlatımın ilk yarısı İngilizce, ikinci yarısının büyük kısmı Türkçe. Ortada 10 dakikalık ara var.

Genel Konular

- MapReduce'un kısa tekrarı: Map, Group by Key, Reduce, arıza yönetimi.
- Büyük veri yazılım yığını (big data stack): HDFS, YARN / Mesos, Hadoop / Spark.
- MapReduce iyileştirmeleri (refinements):
  - Birleştirici (combiner) fonksiyonu.
  - Birleştiricinin şartı: değişme (commutative) ve birleşme (associative) özellikleri.
  - Bölümleme (partition) fonksiyonunu özelleştirme.
- MapReduce uygulamaları ve bulut ortamları.
- Örnek sınav sorusu: asal bölenlerle MapReduce.
- 3. bölüm: Benzer belgeleri bulma problemi ve uygulamaları.
- Üç adım: shingling, MinHashing, yerele duyarlı özetleme (Locality Sensitive Hashing, LSH).
- k-shingle, shingle'ları tamsayıya hash'leme.
- Jaccard benzerliği ve belgelerin 0/1 kolon vektörü olarak gösterimi.
- MinHashing: permütasyonla imza matrisi (signature matrix) oluşturma.

Hocanın Özellikle Vurguladığı Kısımlar

- Bu ders önemli. Dönem projesi tamamen MapReduce programlama modeli üzerine olacak. Projede herkes HDFS kullanacak.
- Sınavda da benzer MapReduce soruları çıkabilir: Verilen Map ve Reduce tanımıyla girdiden çıktıyı bulma.
- MapReduce'ta en çok zaman alan iş, ara çiftlerin Map sunucularından Reduce sunucularına kopyalanmasıdır. Birleştirici bu trafiği azaltır.
- Birleştirici, programcının yazdığı üçüncü fonksiyondur. Reduce ile aynı hesabı yapar ama Map sunucusunda, yerel veriler üzerinde çalışır.
- Birleştirici yalnız Reduce fonksiyonu değişme ve birleşme özelliklerine sahipse eklenebilir. Toplama uygundur. Ortalama ve medyan doğrudan uygun değildir.
- Bölümleme fonksiyonunu değiştirmek programı hızlandırmaz. Problemi kurduğun mantığa göre daha etkili çözmeyi sağlar.
- Belgeler arası benzerlik, satır ve belge sayısı yüz binler veya milyonlar mertebesinde olunca doğrudan hesaplanamaz. Bu yüzden belgeler daha küçük bir imza matrisiyle, bilgi kaybetmeden temsil edilir.
- İmza matrisindeki iki kolonun benzerliği, girdi matrisindeki Jaccard benzerliğine yakınsar. Hash fonksiyonu sayısı arttıkça fark azalır.

Kısa Tekrar Notları

- Programcı `map`, `reduce` ve isteğe bağlı `combine` yazar. Gruplama, taşıma, zamanlama ve arıza yönetimi kütüphanenin işidir.

- Map çıktısı hash ile bölümlenir, yerel diske küçük dosyalar olarak yazılır. Ana düğüm Reducer'a haber verir. Reducer dosyayı kopyalar, gruplar ve Reduce'u çalıştırır.
- Arıza kuralları:
  - Map işçisi bozulursa: Tamamlanmış ve süren Map görevleri idle yapılıp başka sunucuda tekrarlanır.
  - Reduce işçisi bozulursa: Yalnız süren görevler tekrarlanır. Tamamlananların çıktısı HDFS'tedir.
  - Tek ana düğüm bozulursa: Tüm MapReduce görevi iptal edilir.
- Varsayılan bölümleme: `hash(key) mod R`. Özel örnek: `hash(hostname(URL)) mod R`. Aynı host'a ait tüm URL'ler aynı Reducer'a gider.
- k-shingle (k-gram): Belgede geçen ardışık k karakterlik dizi. Belge, shingle'larının kümesiyle temsil edilir.
- Jaccard benzerliği: $\text{sim}(C_1, C_2) = \frac{|C_1 \cap C_2|}{|C_1 \cup C_2|}$
- Kolon vektörü gösteriminde: $\text{sim} = \frac{a}{a+b+c}$. Burada a: iki kolonda da 1 olan satır sayısı. b ve c: yalnız birinde 1 olan satır sayıları.
- MinHash: Satırların rastgele bir permütasyonunda kolonun ilk 1 değerini aldığı satırın sırası: $h_\pi(C) = \min_\pi \pi(C)$.
- İmza matrisi: Her hash fonksiyonu (permütasyon) için bir satır, her belge için bir kolon.
- Temel özellik: $P[h_\pi(C_1) = h_\pi(C_2)] = \text{sim}(C_1, C_2)$.
- İmzaların benzerliği: İki imza kolonunun aynı değeri aldığı hash fonksiyonlarının oranı.

Detaylı Açıklamalar

MapReduce tekrarı

- Girdi çiftlerinde anahtar bir veri parçasının kimliği, değer parçanın kendisidir.
- Kelime sayma örneği:
  - Map her kelime için (kelime, 1) üretir.
  - Kütüphane aynı anahtarları gruplar.
  - Reduce değerleri toplar.
- Kütüphane (Hadoop veya Spark) indirildiğinde Map ve Reduce fonksiyonlarının imzası (signature) hazır gelir. Programcı yalnız içlerini yazar.
- Daha önce görülen örnekler: kelime sayma, 5 kelimelik dizileri sayma, bir sitedeki sayfaların toplam boyutu.
- Genelde 7 veri parçası varsa 7 Map görevi olur. Reduce görevi sayısı Map sayısından azdır.

Büyük veri yazılım yığını

- Dağıtık dosya sistemi: HDFS. Dosyayı parçalara böler ve HDFS düğümlerine dağıtır.
- Kaynak yönetimi ve izleme: Mesos, YARN. Hangi sunucuların çalıştığını ve IP adreslerini izler.
- Veri işleme: Hadoop, Spark ve MapReduce'u gerçekleyen diğer açık kaynak kütüphaneler.
- Üstte uygulama: Map ve Reduce fonksiyonlarını içerir. Kütüphane bunları HDFS üzerinde çalıştırır.
- Hive ve Pig gibi kütüphaneler arka planda Hadoop MapReduce'u kullanır, SQL benzeri arayüz sunar.
- Google'ın da kendi MapReduce gerçeklemesi var.
- Amazon ve Google bulutlarında bu kütüphaneler kurulu hâlde bulunur. Öğrenci hesabıyla (hocanın hatırladığı kadarıyla yaklaşık 750 saat) MapReduce işleri çalıştırılabilir.

Birleştirici (combiner)

- Sorun: Terabaytlarca metin üzerinde kelime sayarken ara çift sayısı çok büyük olur. Bunları aynı veri merkezinde bile bir sunucudan diğerine taşımak uzun sürer.
- Çözüm: Map sunucusunda, aynı anahtarlı değerleri önceden birleştirmek. Ağdan (anahtar, değer listesi) yerine (anahtar, kısmi toplam) gider.
- Slayttaki örnek:

  - İki Map sunucusu, değerleri harf olan çiftler üretiyor. Amaç her harfin sayısını bulmak.
  - Birleştirici olmadan: 6 + 6 = 12 ara çift Reducer'a taşınır.
  - Birleştirici ile: Her sunucuda harf başına tek çift kalır. Örneğin bir sunucuda (B, 2), (D, 2) gibi. Toplam 4 + 4 = 8 çift taşınır.
  - Reducer gelen kısmi toplamları toplar. Örnek: B için bir sunucudan 2, diğerinden 1 gelir. Sonuç 3.
- 12'den 8'e düşüş küçük görünür. Ama yüz binlerce ara çiftte büyük kazanç sağlar.

Birleştiricinin şartı

- Reduce fonksiyonu değişme (commutative) ve birleşme (associative) özelliklerine sahip olmalıdır.
- Birleşme: Üç sayıyı farklı gruplayarak toplamak sonucu değiştirmez: $(a + b) + c = a + (b + c)$.
- Değişme: Sırayı değiştirmek sonucu değiştirmez: $a + b = b + a$.
- Toplama (sum): İki özellik de var. Birleştirici kullanılabilir.
- Ortalama (average): Önce ilk iki sayının ortalaması alınıp sonra üçüncüyle ortalaması alınırsa sonuç, gerçek ortalamadan farklı çıkabilir. Doğrudan birleştirici olarak kullanılamaz.
- Medyan (median): Aynı şekilde doğrudan kullanılamaz.
- Kullanılacak her fonksiyon (çarpma, bölme, istatistiksel fonksiyonlar) için bu iki özellik kontrol edilmelidir.

Bölümleme (partition) fonksiyonu

- Varsayılan olarak kütüphane ara anahtarı bir hash fonksiyonuna sokar. Çıkan Reducer numarasına göre çifti o sunucuya gönderir.
- Programcı bu fonksiyonu kendi fonksiyonuyla değiştirebilir.
- Örnek: Host başına toplam sayfa boyutu.
  - Amaç: Aynı host adına sahip tüm URL'lerin aynı Reducer'a gitmesi.
  - Özel fonksiyon: `hash(hostname(URL)) mod R`.
  - Varsayılanda tam URL hash'lenirdi.

Örnek soru: Asal bölenlerle MapReduce

- Girdi: Tamsayılar.
- Map: i tamsayısını alır. i'yi bölen her asal p için (p, i) çifti üretir. Örnek: 12 → (2, 12), (3, 12).
- Reduce: Her anahtarın değerlerini toplar.
- Girdi: 15, 21, 24, 30, 49.

Girdi	Asal bölenler	Map çıktısı
15	3, 5	(3,15), (5,15)
21	3, 7	(3,21), (7,21)
24	2, 3	(2,24), (3,24)
30	2, 3, 5	(2,30), (3,30), (5,30)
49	7	(7,49)

- Gruplama ve Reduce:
  - 2: 24 + 30 = 54
  - 3: 15 + 21 + 24 + 30 = 90
  - 5: 15 + 30 = 45
  - 7: 21 + 49 = 70
- Seçeneklerde çıktıda bulunabilecek çift: (5, 45).
- Dönem projesindeki programlama da buna benzer olacak: Verilen probleme göre Map ve Reduce içeriğini belirle, gerçekle, birden fazla sunucuda çalıştır.

Benzer belgeleri bulma: uygulamalar

- Benzer kelimeler içeren sayfaları bulup konuya göre sınıflandırmak. Örnek: Başlıksız 1000 Wikipedia sayfasını spor, eğitim, siyaset gibi konulara ayırmak.
- Netflix'te film zevki benzer kullanıcıları bulmak. Benzer kullanıcılar gruplanırsa öneri bir kişiye değil, aynı zevkteki birçok kişiye yapılabilir.
- Tersi: Benzer hayran kitlesine sahip filmleri bulmak. Bu da öneri sağlar.
- Varlık çözümleme (entity resolution): Facebook, Instagram, Twitter'da aynı kişiye ait profilleri bulmak.
- Ortak metni çok olan belge çiftlerini bulmak:
  - Ayna siteler (mirror sites): Arama motorları aynı içeriği yalnız bir kez indekslemek ister.
  - İntihal (plagiarism): Başka belgeden alınmış büyük alıntıları yakalamak.
  - Birçok sitede yayımlanan benzer haber makalelerini bulmak.

Üç adım

- Shingling: Belgeyi kümeye dönüştürür.
- MinHashing: Büyük kümeleri, benzerliği koruyan kısa imzalara dönüştürür.
- LSH: İmzalara bakarak benzer olma olasılığı yüksek aday çiftleri (candidate pairs) bulur.

Shingling

- k-shingle (doğal dil işlemede k-gram, n-gram): Belgede geçen k karakterlik dizi.
- Örnek: $k = 2$, belge $D_1 = abcab$. 2-shingle kümesi: $S(D_1) = \{ab, bc, ca\}$.
  - ab ikinci kez geçse de bir kez yazılır. Çünkü bu bir kümedir.
- Fikir: Sezgisel olarak benzer belgelerin ortak shingle'ı çoktur.
- Bir kelimeyi değiştirmek yalnız o kelimeye yakın shingle'ları etkiler.
- Slayt örneği ($k = 3$):
  - D_1 = "The dog which chased the cat"
  - D_2 = "The dog that chased the cat"
  - "which" yerine "that" konunca D_1'in 7 shingle'ı değişir: `wh`, `whi`, `hic`, `ich`, `ch`, `g w`, `h c`. Diğer tüm 3-shingle'lar aynı kalır.
- Belleği azaltmak için shingle'lar bir hash fonksiyonuyla tamsayıya (örneğin 4 bayt) dönüştürülür. Belge, bu tamsayıların kümesiyle temsil edilir.

Jaccard benzerliği

- İki kümenin kesişiminin eleman sayısı bölü birleşiminin eleman sayısıdır.
- Slayt örneği: Kesişimde 3, birleşimde 8 eleman var. Benzerlik $3/8$.
- Oran yüksekse belgeler daha benzerdir.

Belgeleri 0/1 kolon vektörü olarak göstermek

- Tüm belgelerden çıkan farklı shingle'lar evrensel küme (universal set) olarak alınır. Her shingle bir satırdır.
- Her belge bir kolondur. Shingle o belgede varsa 1, yoksa 0 yazılır.
- Matrisin satır sayısı, tüm belgelerdeki farklı shingle sayısıdır.
- Slayt örneği: 6 satırlık iki kolon C_1, C_2. İkisinde de 1 olan 2 satır, en az birinde 1 olan 5 satır var. Benzerlik $2/5$.
- Satır türleri:
  - a: iki kolonda da 1.
  - b: C_1'de 1, C_2'de 0.
  - c: C_1'de 0, C_2'de 1.
  - d: ikisinde de 0 (benzerliğe katılmaz).
- $\text{sim}(C_1, C_2) = a/(a + b + c)$.
- Sorun: Satır sayısı 100.000'ler, belge sayısı 100.000'ler mertebesinde olunca tüm çiftleri karşılaştırmak hem zaman hem hesaplama açısından çok pahalıdır.

MinHashing

- Amaç: Girdi matrisini, aynı kolon sayısına ama çok daha az satıra sahip bir imza matrisine dönüştürmek.
- Her hash fonksiyonu satırların rastgele bir permütasyonudur.
- Bir kolonun hash değeri: Permütasyon sırasında, kolonun 1 olduğu ilk satırın sırası.
- Her hash fonksiyonu imza matrisinde bir satır üretir. 3 hash fonksiyonu varsa 3 satır, 100 varsa 100 satır olur.
- Slayttaki örnek (7 shingle, 4 belge, 3 permütasyon):

```
Permütasyonlar        Girdi matrisi     İmza matrisi M
 π3 π2 π1             C1 C2 C3 C4       C1 C2 C3 C4
 1  4  3              1  0  1  0        2  1  2  1    (π1)
 3  2  4              1  0  0  1        2  1  4  1    (π2)
 7  1  7              0  1  0  1        1  2  1  2    (π3)
 6  3  6              0  1  0  1
 2  6  1              0  1  0  1
 5  7  2              1  0  1  0
 4  5  5              1  0  1  0
```

- Her permütasyon kolonundaki sayı, o satırın permütasyondaki sırasıdır.
- Örnek hesap, π_1 ve C_1:
  - Sırası 1 olan satır 5. satırdır. C_1'de 0, devam edilir.
  - Sırası 2 olan satır 6. satırdır. C_1'de 1. Sonuç 2.
- Örnek hesap, π_2 ve C_3: Sıra 1, 2 ve 3'teki satırlarda C_3 0, sıra 4'teki satırda 1. Sonuç 4.

İmza benzerliği ile Jaccard benzerliğinin karşılaştırılması

	1-3	2-4	1-2	3-4
Kolon/Kolon (Jaccard)	0,75	0,75	0	0
İmza/İmza	0,67	1,00	0	0

- 2-4 için Jaccard: İkisinde de 1 olan 3 satır, en az birinde 1 olan 4 satır. $3/4 = 0{,}75$.
- 2-4 imzaları tamamen aynı. İmza benzerliği 1.
- 1-3 imzaları 3 hash fonksiyonunun 2'sinde aynı. İmza benzerliği $2/3 \approx 0{,}67$.
- 1-2 imzalarında hiç eşleşme yok. İki benzerlik de 0.
- Hash fonksiyonu sayısı artarsa imza benzerliği Jaccard benzerliğine daha da yaklaşır.
- Kazanç: Milyonlarca satırlık iki kolon yerine birkaç satırlık iki vektör karşılaştırılır.

Soru-cevap: Girdi matrisinden imza matrisine geçiş

- Bir öğrenci geçişi kaçırdığını söyledi, hoca adım adım tekrar anlattı:
  - Her belgedeki tüm k karakterlik diziler çıkarılır. Tüm belgelerdeki farklı diziler alt alta yazılır.
  - Her belge, bu dizileri içerip içermemesine göre 0/1 kolon vektörü olur.
  - Her hash fonksiyonu kendi permütasyonunu kullanır. Permütasyonda ilk 1 bulunan sıra, imza matrisinin o hash satırına ve o belgenin kolonuna yazılır.
- Öğrenci örnekleri doğru çözdü:
  - π_1, C_2 için: sıra 1'deki satırda 1 var. Sonuç 1.
  - π_3, C_2 için: sıra 1'de 0, sıra 2'de 1. Sonuç 2.

Ders sonrası notlar

- Hoca "MinHashing" adının birden fazla hash fonksiyonu kullanılmasından geldiğini söyledi. Ad aslında permütasyondaki en küçük (minimum) sıranın alınmasından gelir: $h_\pi(C) = \min_\pi \pi(C)$.
- Hoca birleştiricinin şartını anlatırken "communicative" dedi. Doğru terim "commutative" (değişme özelliği).

- Ortalama doğrudan birleştirici olamaz. Ama birleştirici (toplam, adet) çifti üretirse, Reducer toplamları ve adetleri ayrı ayrı toplayıp en sonda bölerek ortalamayı doğru bulur.
- Hoca bir kelime değişince "2K karakter öncesi ve sonrası" shingle'ların etkilendiğini söyledi. Örnekte etkilenen shingle'lar, değişen kelimeyle çakışan, en fazla $k-1$ karakter önce başlayan shingle'lardır. Slayttaki 7 değişen shingle bununla tutarlı.
- "Kesişimin olasılığı" ifadesinin doğru hâli: Rastgele bir permütasyonda iki kolonun MinHash değerlerinin eşit olma olasılığı, Jaccard benzerliğine eşittir.
- Altyazının bozuk olduğu yerler (00:43:47 civarı tek kelimelik bloklar, 00:48:44 civarı tekrar, 01:28:45 civarı) özetlenmedi.
- Kalınan yer: 3. bölüm, 12. slayt, MinHashing'in gerçeklenmesi (implementation). Sonra LSH işlenecek.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.